

XaaDir App

Final Project Report — Attendance Management System

React JS + Tailwind CSS • ASP.NET Core Web API • ADO.NET • SQL Server

Project Name	XaaDir Attendance Management System
Project Type	Full-stack Web Application
Main Users	Admin, Teacher, TeacherAdmin
Status	Database + Backend + Frontend completed; final testing and deployment remain

1. Executive Summary

XaaDir App is a full-stack attendance management system designed for a secondary school environment. The system allows administrators to manage users, classes, subjects, students, attendance records, and reports. Teachers can mark class attendance based on their assigned subjects. The latest correction changed attendance from single-student marking into a class-based workflow, which is closer to real school attendance practice.

- Admin manages the whole system and has access to all data.
- Teachers can view their subjects, load class students, mark Present/Late, and save full class attendance.
- Unticked students are automatically saved as Absent.
- Reports include attendance totals, status charts, and student percentage reports.
- TeacherAdmin role allows a teacher to receive admin access without changing the pure Admin account.

2. Project Objectives

- Build a realistic attendance management system aligned with the course project guidelines.
- Use SQL Server with normalized tables, primary keys, foreign keys, and sample data.
- Use ASP.NET Core Web API as the backend and ADO.NET for database operations.
- Use React JS and Tailwind CSS to build a responsive modern frontend.
- Provide CRUD operations, search, validation, role-based access, reports, and API integration.
- Push all source code to GitHub with clear folder structure and documentation.

3. Technology Stack

Layer	Technology	Purpose
Frontend	React JS + Vite + Tailwind CSS	User interface, dashboards, forms, CRUD pages, reports
State Management	Context API	Authentication state and user role access
API Client	Axios	HTTP requests from React to backend
Backend	ASP.NET Core Web API	API layer, routing, validation, business rules
Database Access	ADO.NET / Microsoft.Data.SqlClient	SQL connection, commands, readers, CRUD queries
Database	SQL Server / XaaDirDB	Persistent data storage
API Testing	Swagger	Backend endpoint testing
Version Control	Git + GitHub	Source control and final submission repository

4. Final Folder Structure

Folder	Content
backend/XaaDirApi	ASP.NET Core Web API source code

frontend/xaadir-client	React JS + Tailwind CSS frontend source code
database	SQL scripts including XaaDirDB.sql and role update script
documentation	Project report, table schema, ERD overview, final presentation
README.md	Main GitHub project description

5. Database Design

The database is named XaaDirDB. It uses five main tables that satisfy the requirement for a 3–5 table relational database design.

Table	Purpose	Key Relationships
Users	Stores admin, teacher, and teacher-admin login accounts	Users.UserId → Subjects.TeacherUserId; Users.UserId → Attendance.MarkedByUserId
Classes	Stores Form 1 to Form 4	Classes.ClassId → Students.ClassId; Classes.ClassId → Attendance.ClassId
Subjects	Stores school subjects and teacher assignments	Subjects.SubjectId → Attendance.SubjectId
Students	Stores student information and assigned class	Students.StudentId → Attendance.StudentId
Attendance	Stores attendance result for every student per class session	References Students, Classes, Subjects, Users

- Initial sample data includes 1 Admin, 5 Teachers, 4 Classes, 12 Subjects, 200 Students, and attendance sample records.
- The new SQL update supports TeacherAdmin while keeping Admin protected as a pure role.
- Attendance records use Present, Absent, and Late statuses.

6. Backend Design

The backend is implemented as an ASP.NET Core Web API. ADO.NET is used directly through SqlConnection, SqlCommand, SqlDataReader, ExecuteReader, ExecuteNonQuery, and ExecuteScalar.

Backend Area	Files / Features
Controllers	AuthController, UsersController, ClassesController, SubjectsController, StudentsController, AttendanceController, ReportsController, DashboardController
Models	User, LoginRequest, LoginResponse, ClassModel, Subject, Student, Attendance, ClassAttendanceRequest, RoleHelper
Repositories	ADO.NET CRUD and report logic
Data	DbConnectionFactory for SQL Server connection string
Swagger	Used to test endpoints before connecting frontend

7. Frontend Design

The frontend is implemented using React JS with Tailwind CSS. It provides a modern responsive interface with role-based dashboards and navigation.

Frontend Requirement	Implementation
React Router	Routes for login, dashboards, CRUD pages, attendance, reports, and about page
Context API	AuthContext stores login user and role access
Reusable Components	Button, Badge, DataTable, Modal, Sidebar, Navbar, StatCard, SearchBox, FormInput, AttendanceBars
API Integration	Axios apiClient connected to http://localhost:5168/api
Validation	Reusable validation functions for users, students, subjects, classes, and attendance
Responsive Design	Tailwind CSS responsive grids, cards, tables, and mobile sidebar
Toast Notifications	React Toastify feedback for login, save, delete, and errors

8. Corrected Class Attendance Workflow

- Teacher selects Class, Subject, and Date.
- Teacher clicks Load Students to display all active students in that class.
- Teacher ticks Present for students who attended normally.
- Teacher ticks Late for students who arrived late.
- Students with neither Present nor Late selected are automatically saved as Absent.
- Teacher clicks Save Class Attendance to save all students at once.
- The system shows total students, present, absent, late, and chart percentages.

Student Action	Saved Status
Present checked	Present
Late checked	Late
Nothing checked	Absent

9. Role and Permission Rules

Role	Meaning	Access
Admin	Pure administrator account	Full system access; cannot be changed into another role
Teacher	Normal teacher account	Assigned subjects, class attendance, own reports
TeacherAdmin	Teacher with admin access	Admin access plus teacher capabilities

The system protects the pure Admin account. A teacher can be promoted to TeacherAdmin, but a teacher is not converted into pure Admin. This keeps the role design realistic and controlled.

10. API Endpoint Summary

Feature	Main Endpoints
Login	POST /api/Auth/login
Users	GET/POST/PUT/DELETE /api/Users
Classes	GET/POST/PUT/DELETE /api/Classes
Subjects	GET /api/Subjects, GET /api/Subjects/teacher/{teacherUserId}
Students	GET /api/Students, GET /api/Students/class/{classId}, CRUD endpoints
Class Attendance	POST /api/Attendance/mark-class, GET /api/Attendance/class
Student Percentage	GET /api/Attendance/student-percentages/class/{classId}
Reports	GET /api/Reports/admin/summary, GET /api/Reports/teacher/{id}/summary

11. Testing Plan

- Run SQL Server and verify XaaDirDB exists.
- Run backend using dotnet run and test <http://localhost:5168/api/Classes>.
- Use Swagger to test login, CRUD, mark-class attendance, and reports endpoints.
- Run frontend using npm run dev and open <http://localhost:5173>.
- Login as Admin and verify admin menu, CRUD pages, and reports.
- Login as Teacher and verify only assigned subjects and teacher reports appear.
- Test Class Attendance by loading a class, marking Present/Late students, and saving attendance.
- Check Reports page for charts and student percentage report.

12. Final Status and Remaining Work

Phase	Status
Database	Completed and pushed to GitHub
Backend	Completed, connected to SQL Server, class attendance fixed
Frontend	Completed with React JS + Tailwind CSS
Documentation	This report and presentation generated
GitHub	Push latest fixed frontend/backend/database after testing
Deployment	Optional/final requirement if instructor asks for public URL

Conclusion: XaaDir App is functionally ready after final local testing and GitHub push. If the course rubric requires live deployment, the next step is deploying the frontend and backend, then updating the README with public URLs.